

Kusama Yayoi 반응형 웹사이트

반응형 웹사이트 구축 프로젝트

프로젝트 전체 구성

본 발표는 웹퍼블리싱의 전 과정을 체계적으로 다룹니다. 기획부터 디자인, 개발, 그리고 최종 구현까지의 모든 단계를 상세히 살펴보겠습니다.

01

프로젝트 개요

제작 배경과 목표 설정

03

설계 단계

사이트맵 및 구조 설계

05

와이어프레임

레이아웃 구조 설계

07

HTML 마크업

시맨틱 구조 구현

09

반응형 웹 구현

다양한 디바이스 최적화

02

분석 단계

벤치마킹 및 요구사항 분석

04

UI/UX 디자인

디자인 컨셉 및 시스템

06

피그마 프로토타입

인터랙티브 목업 제작

08

CSS 스타일링

비주얼 디자인 적용

10

JavaScript 구현

인터랙션 및 동적 기능

프로젝트 개요

쿠사마 야요이 작품 전시 반응형 웹사이트

프로젝트 기본 정보

제작 기간: 2025.07.18 ~ 2025.10.27

작업 범위: UI/UX 디자인 & 퍼블리싱

사용 기술:

- HTML5 - 시맨틱 마크업
- CSS3 - 스타일링 및 애니메이션
- JavaScript - 인터랙션 구현
- Figma - 디자인 프로토타이핑

구현 페이지 구성

1. **Main** 페이지 - 작가 소개, 영상 썸네일, CTA 버튼
2. **Profile** 페이지 - 프로필 이미지, 스크롤 애니메이션
3. **Works** 페이지 - 작품 그리드, 페이지네이션, 모달 팝업
4. **Museum** 페이지 - Visit(방문안내, 지도), Exhibition(전시정보)
5. **Contact** 페이지 - 문의 폼, 입력 필드, 제출 기능

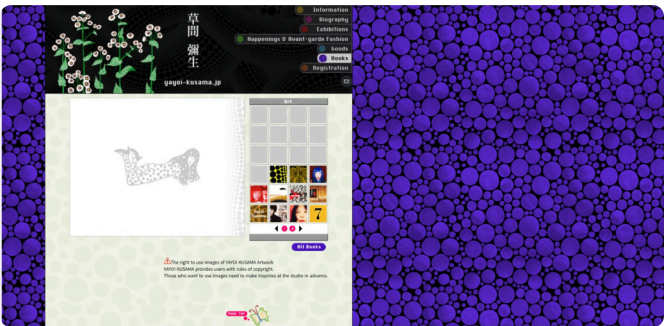
반응형 디자인: Mobile, Tablet, Desktop

기존 홈페이지 문제점 및 목표설정

웹페이지 선택이유

웹 퍼블리셔 과정을 듣기 전, 쿠사마 야요이 작가의 전시회를 찾아보면서 공식 웹사이트를 이용한 적이 있었습니다. 그때 느꼈던 불편함들이 계속 기억에 남아, 이를 직접 개선해보고 싶다는 생각으로 해당 웹페이지를 선정했습니다.

기존 사이트는 구식 디자인과 UI/UX, 한쪽으로 치우친 레이아웃, 그리고 작가 사이트와 미술관 사이트가 분리되어 있어 정보 접근성이 떨어지는 점 등이 문제로 보였습니다. 이러한 부분을 정리하고, 사용자 경험 중심으로 재구성하는 것을 목표로 리디자인 프로젝트를 진행했습니다.



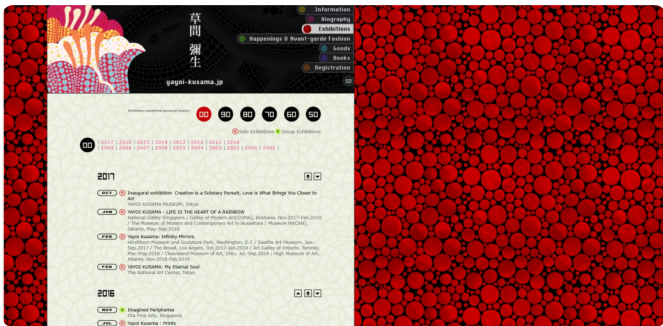
디자인 & UI/UX

문제점

- 테이블 레이아웃 기반의 구식 디자인
- 가독성 낮음

해결방안

- 모던 CSS Grid/Flexbox 레이아웃 도입
- 쿠사마의 작품 세계를 반영한 비주얼 아이덴티티 구축



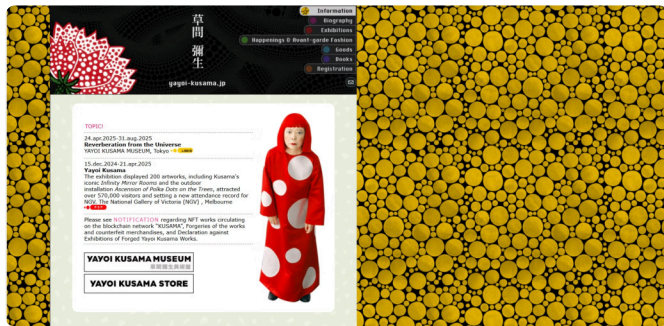
모바일 경험

문제점

- 테이블 레이아웃으로 모바일에서 깨짐

해결방안

- 반응형 디자인 전면 재설계
- 햄버거 메뉴 도입



웹사이트의 분리 문제

문제점

- 사용자가 3개의 다른 사이트를 오가며 정보를 찾아야 함
- 브랜드 통일성 부족
- 사용자 경험 단절
- 내비게이션 혼란

해결방안

- 원스톱 경험 - 모든 정보가 한 곳에
- 통일된 디자인 - 일관된 브랜드 아이덴티티

분석 단계

벤치마킹 및 요구사항 도출

성공적인 웹사이트 제작을 위해 다양한 레퍼런스를 분석하고, UI/UX 및 웹 접근성 요구사항을 명확히 정의했습니다.

백세주 홈페이지

분석 포인트: 큰 틀 및 메인 레이아웃

- 깔끔하고 특색 있는 디자인 구성
- 브랜드 스토리를 통한 감정적 연결
- 이미지 중심 콘텐츠로 시각적 임팩트 강화

국립고궁박물관

분석 포인트: 정보 구조 및 가독성

- 깔끔하고 명확한 레이아웃 설계
- 체계적인 정보 계층 구조
- 사용자 친화적인 네비게이션

서울 국제도서전

분석 포인트: 인터랙션 및 UX

- 애니메이션 효과로 역동적인 경험 제공
- 미미한 콘텐츠를 재미있게 표현
- 사용자 참여를 유도하는 인터랙티브 요소

도출된 핵심 요구사항

UI 요구사항

- 직관적인 네비게이션 구조 설계
- 시각적 계층 구조 명확화
- 일관된 디자인 시스템 적용

웹 표준 & 접근성

- 시맨틱 HTML 마크업 준수
- WCAG 접근성 가이드라인 적용
- 크로스 브라우징 호환성 확보

설계 단계

사이트맵 및 레이아웃 구조

사이트맵 구조

전체 웹사이트는 5개의 주요 섹션으로 구성되며, 각 섹션은 명확한 목적과 하위 페이지를 포함합니다.



Main

홈페이지 - 작가 소개 및 주요 콘텐츠 진입점



Profile

작가 프로필 - 이력 및 연대기



Works

작품 소개 - Paintings, Books



Museum

박물관 정보 - Visit, Exhibition



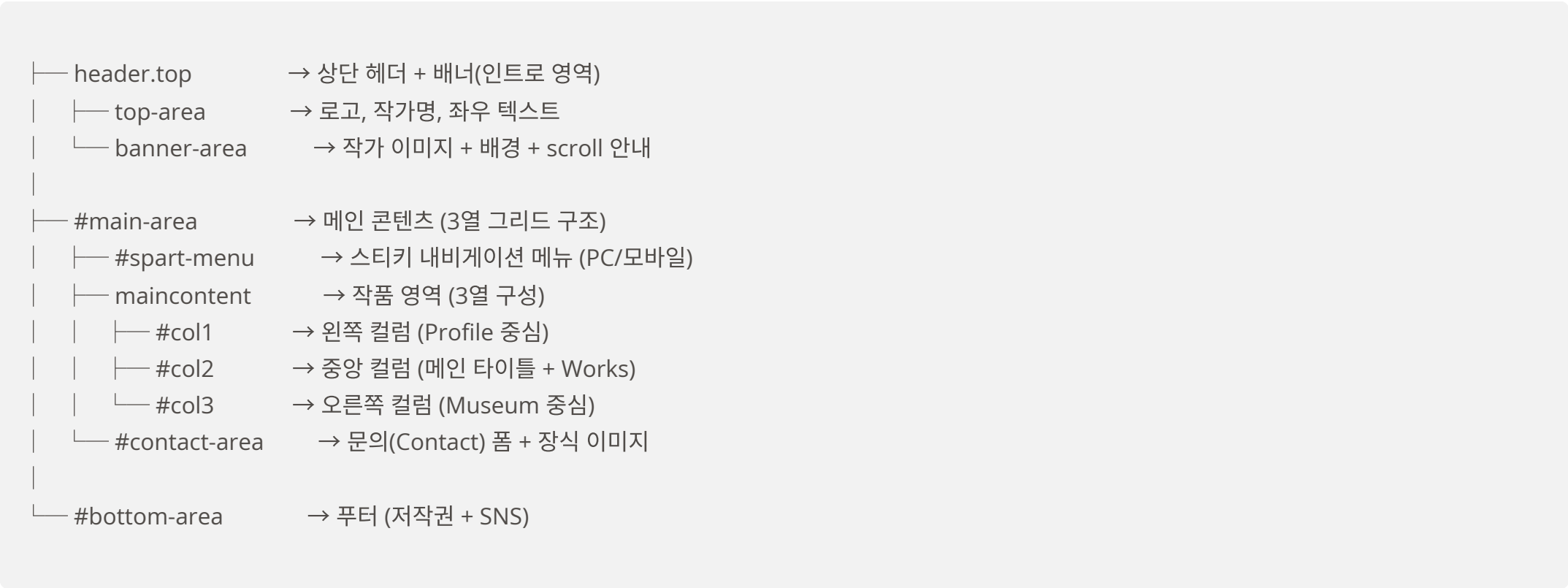
Contact

문의하기 - 폼 및 연락처

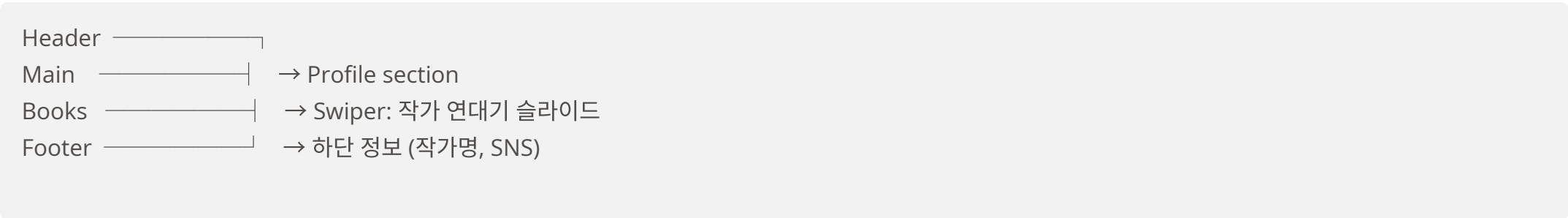
레이아웃 구조 설계

페이지별 특성에 맞춘 최적화된 레이아웃을 설계했습니다. 각 페이지는 독립적인 구조를 가지면서도 일관된 사용자 경험을 제공합니다.

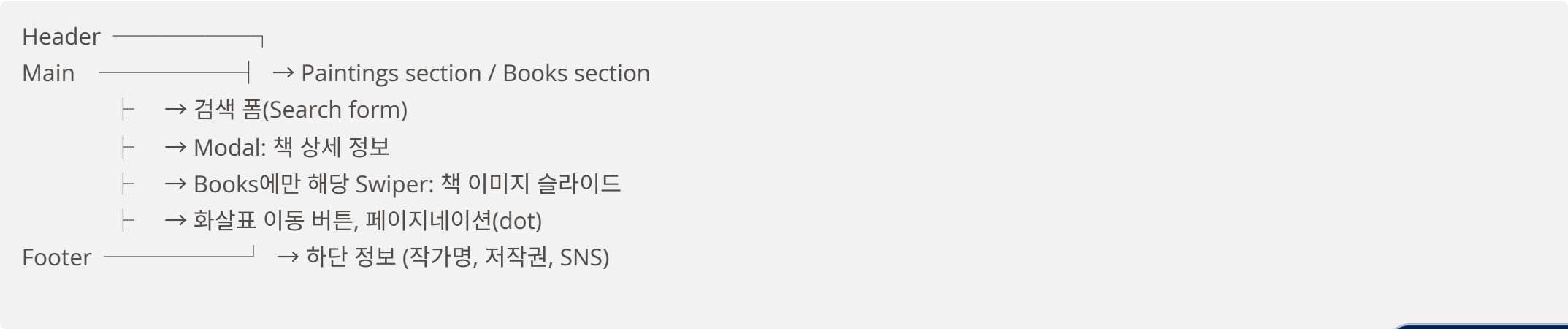
1) Main & Contact 페이지



2) Profile 페이지



3) Works 페이지 - Paintings, Books



4) Museum 페이지 - Visit(Visitinfo, Facilities, about), Exhibition(Current exhibition, Past exhibition)



UI/UX 디자인

디자인 컨셉 및 시스템

"쿠사마 야요이 작가의 독특하고 창의적인 예술 세계를 반영한 현대적이고 미니멀한 디자인"

디자인 핵심 원칙

그리드 기반 구조

정돈된 배열과 체계적인 레이아웃으로 콘텐츠의 가독성과 시각적 균형을 확보합니다.

콘텐츠 중심

불필요한 장식 요소를 배제하고 작품과 정보를 중심으로 디자인을 구성합니다.

기능 단순화

직관적인 인터페이스와 명확한 인터랙션으로 사용자 경험을 최적화합니다.

컬러 시스템



Primary Color

#c52531

강조 및 CTA 요소

쿠사마 야요이의 대표 색상으로 브랜드 아이덴티티를 표현



Secondary Color

#ffffff

배경 및 여백

깔끔하고 미니멀한 느낌을 위한 화이트 베이스



Text Color

#000000

본문 및 제목

높은 대비로 가독성을 극대화

타이포그래피 시스템

HCR Batang

용도: 제목 및 헤딩

전통적이면서도 현대적인 감각의 명조 서체로 작가의 예술적 분위기를 표현합니다. 가독성과 심미성을 동시에 확보하여 중요한 정보를 강조합니다.

Instrument Sans

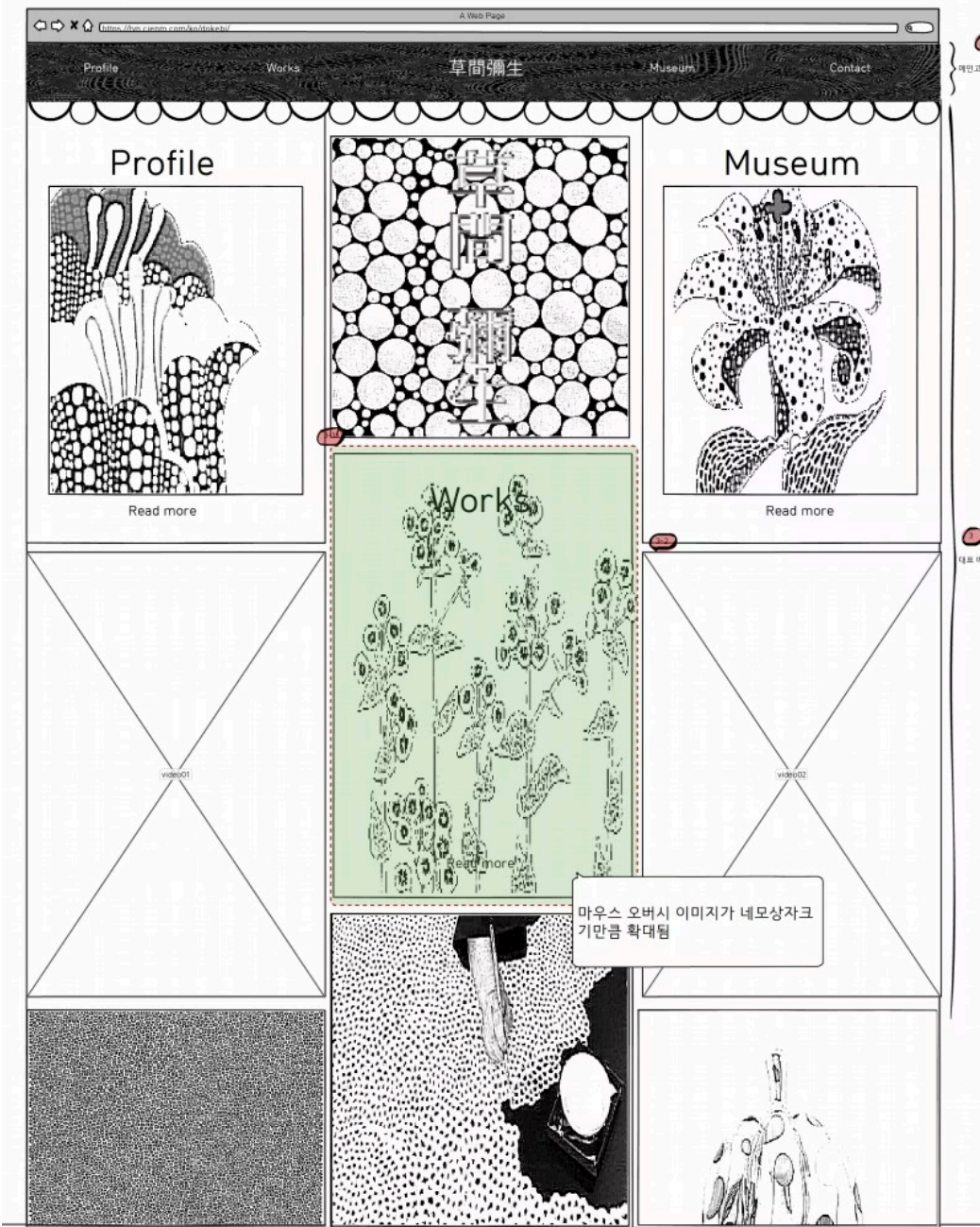
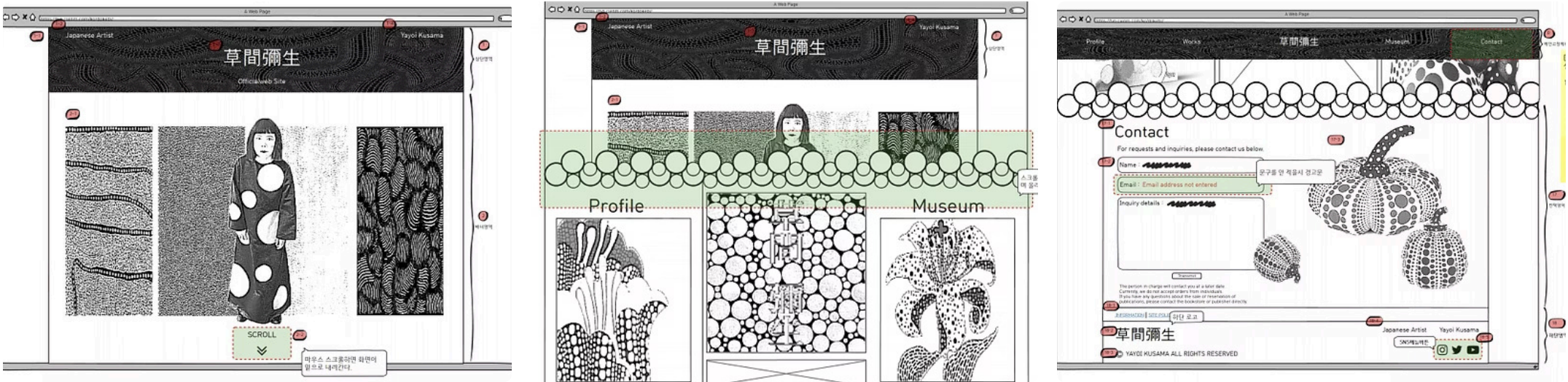
용도: 본문 및 설명 텍스트

깔끔하고 모던한 산세리프 서체로 긴 텍스트의 가독성을 높입니다. 다양한 디바이스에서 최적의 렌더링을 제공합니다.

이러한 디자인 시스템은 쿠사마 야요이 작가의 예술 세계를 디지털 공간에 효과적으로 구현하며, 사용자에게 일관되고 몰입감 있는 경험을 제공합니다.

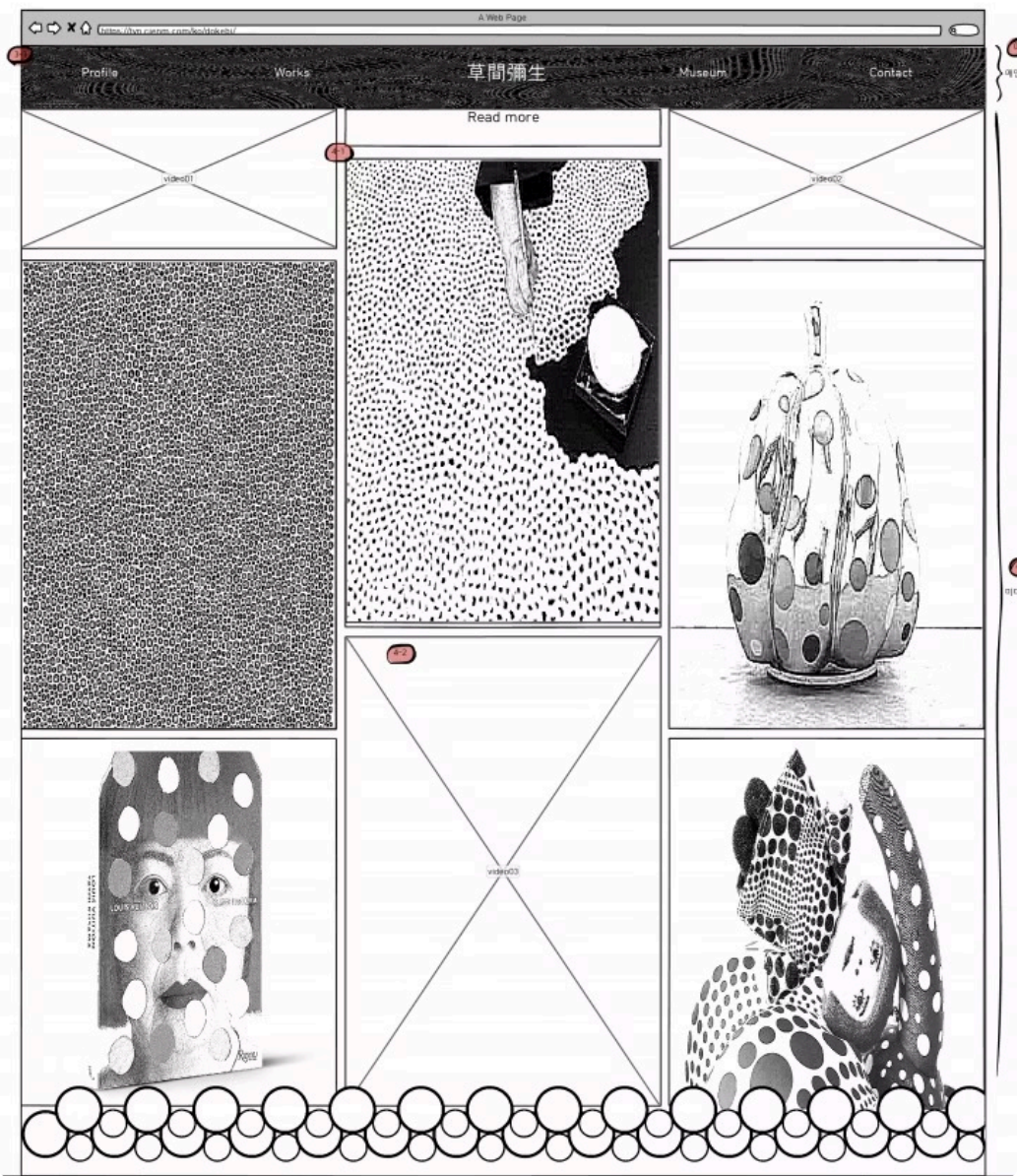
05 와이어프레임 (1)

다양한 디바이스 환경에 최적화된 사용자 경험을 제공하기 위해 데스크탑, 태블릿, 모바일 각 기준에 맞춰 와이어프레임을 설계했습니다.



데스크탑 (1360px 기준)

넓은 화면 공간을 활용하여 정보의 밀도를 높이고, 복잡한 기능에 대한 접근성을 고려했습니다.

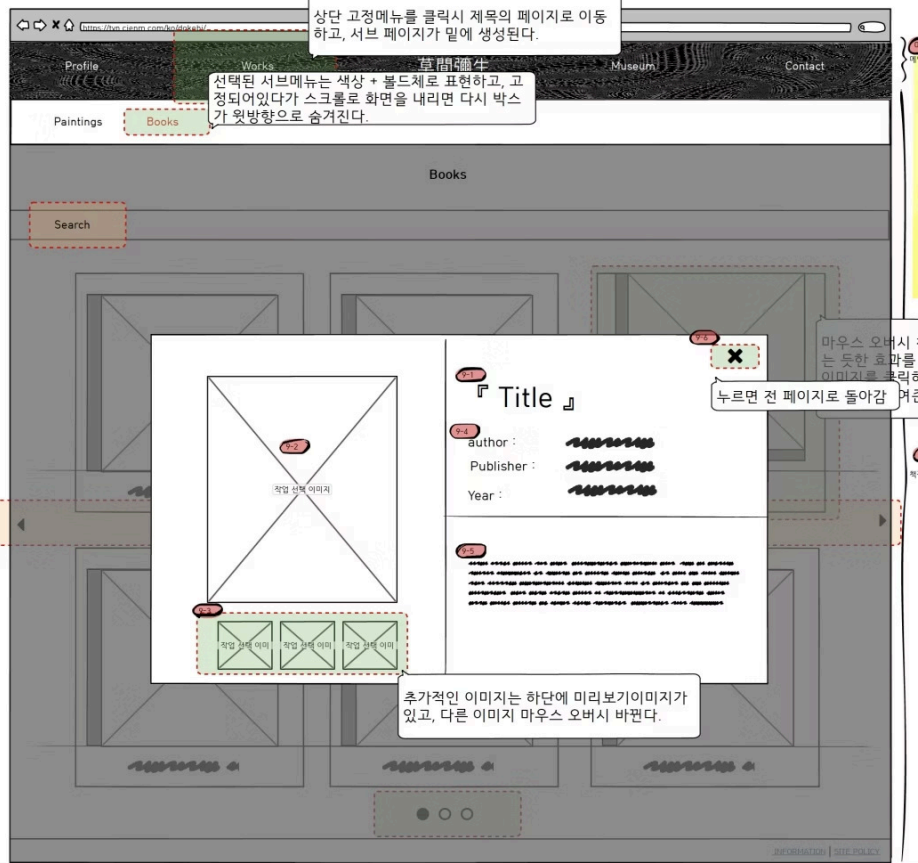
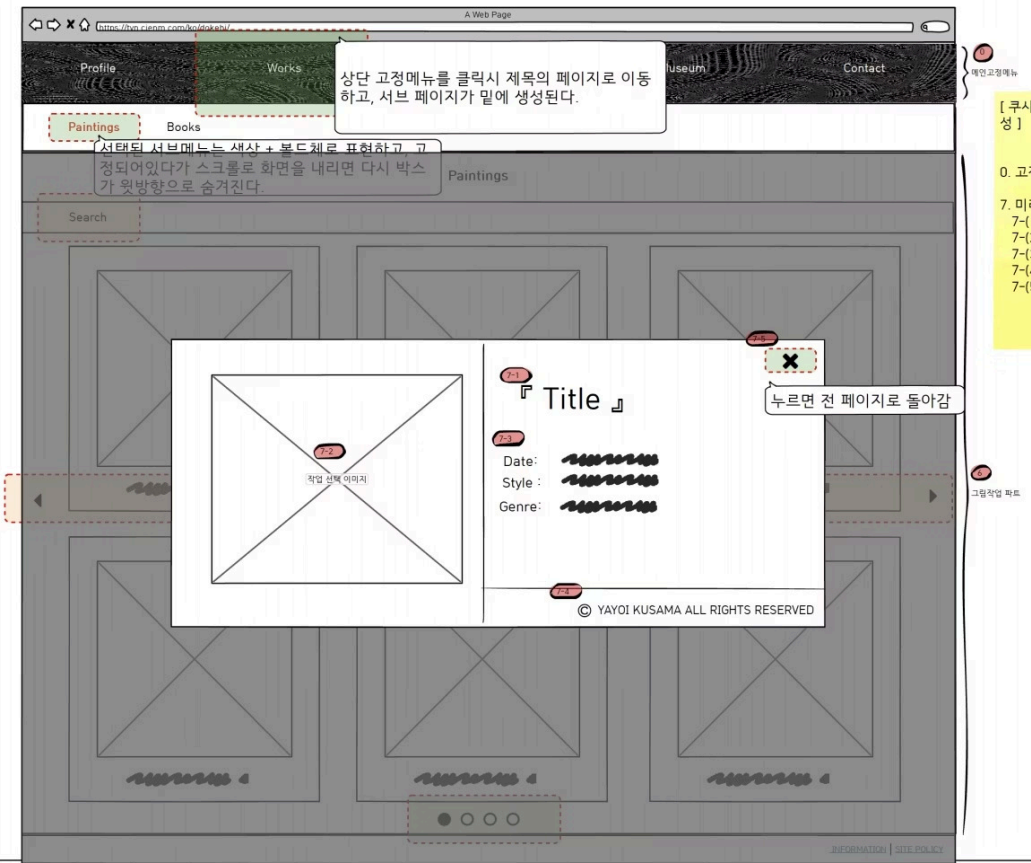
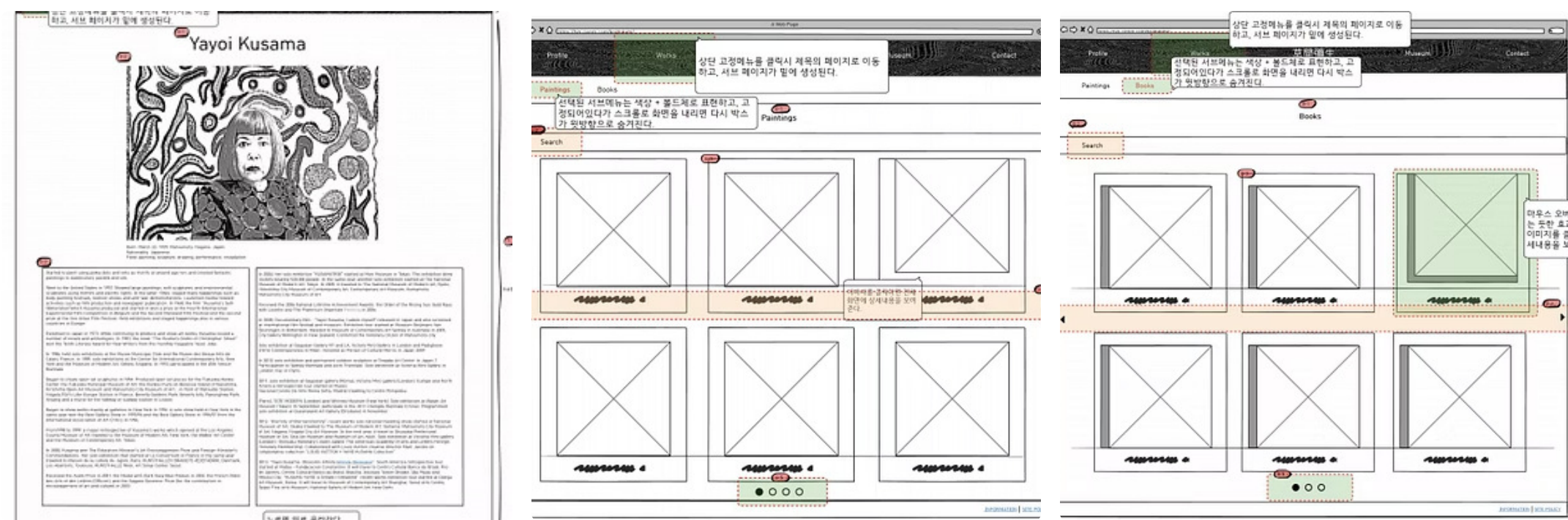


모바일 (360px 기준)

작은 화면에서도 핵심 콘텐츠를 명확하게 전달하고, 한 손 조작 편의성을 고려한 최적화된 디자인을 적용했습니다.

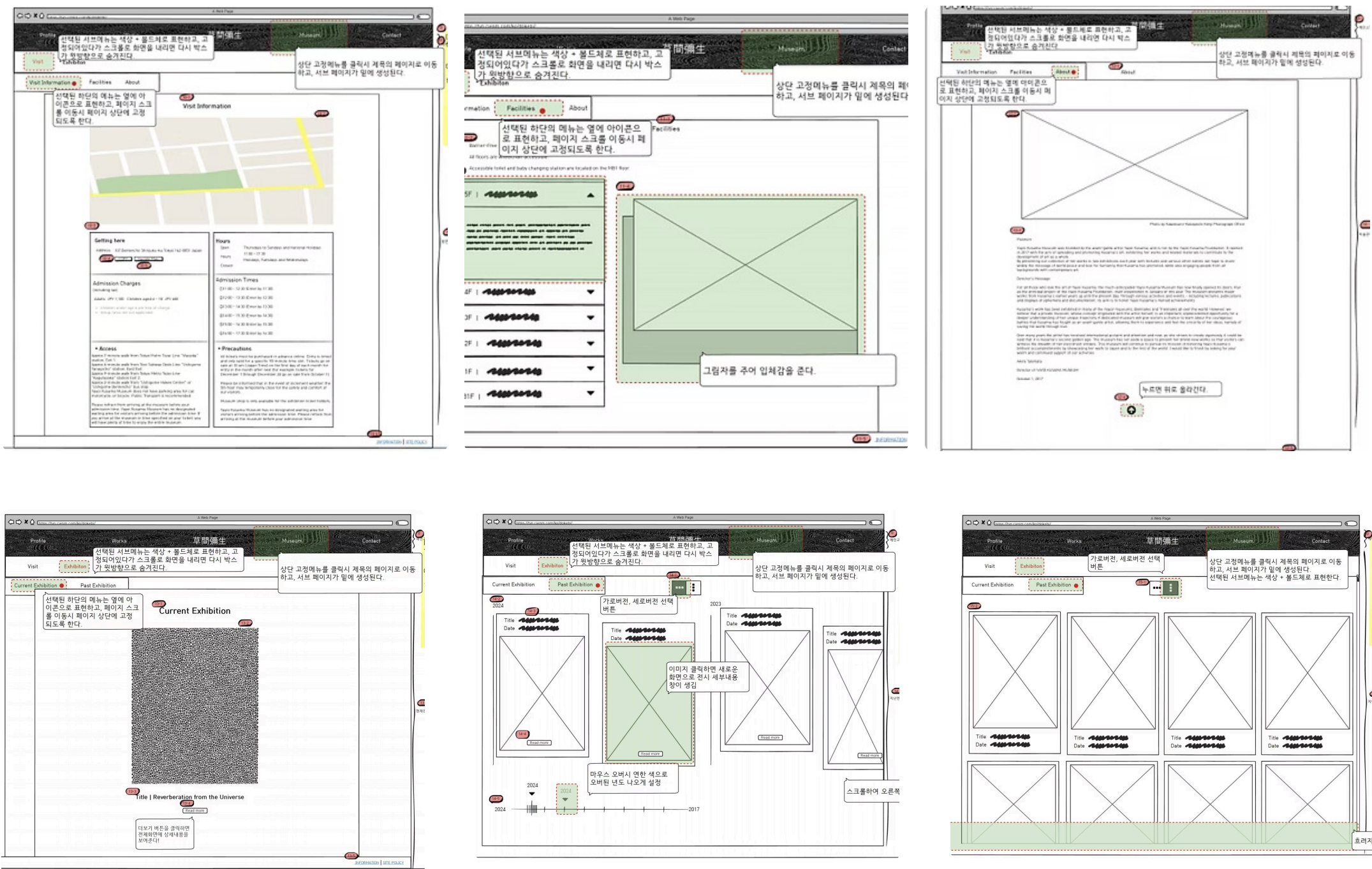
05 와이어프레임 (2)

다양한 디바이스 환경에 최적화된 사용자 경험을 제공하기 위해 데스크탑, 모바일 각 기준에 맞춰 와이어프레임을 설계했습니다.



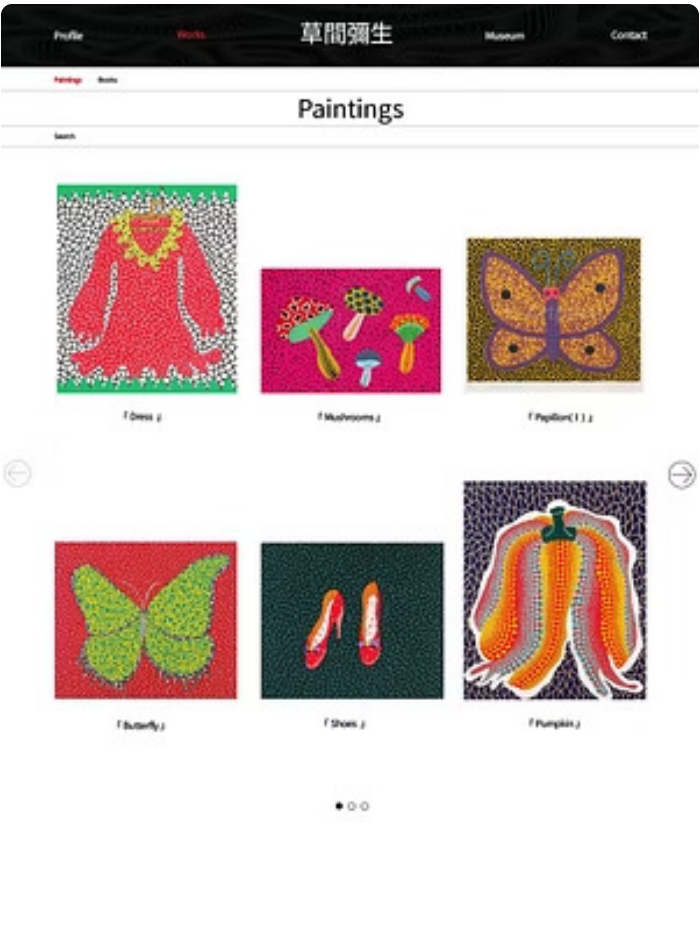
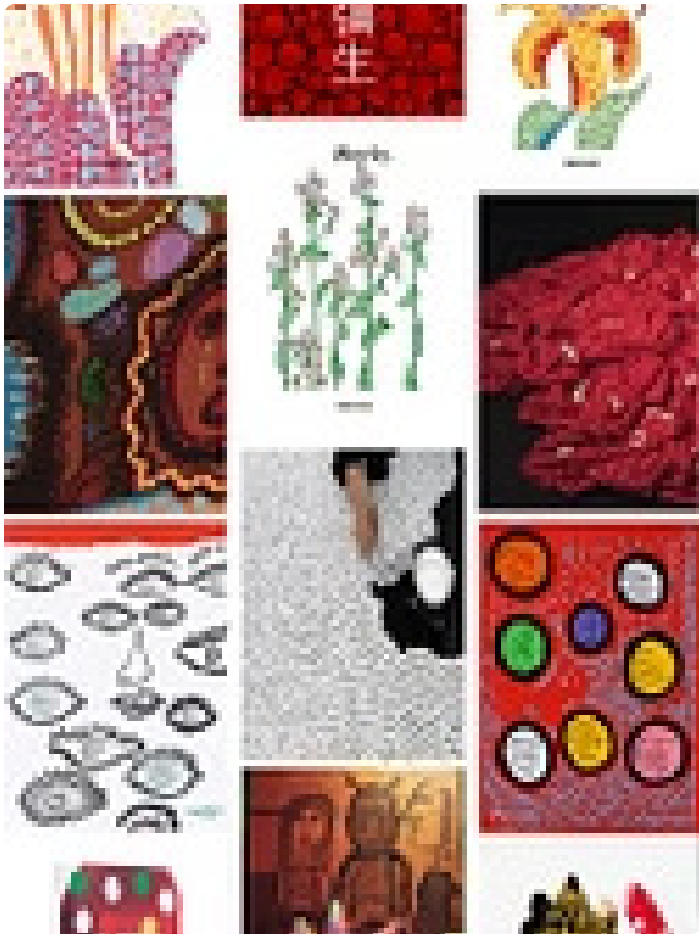
05 와이어프레임 (3)

다양한 디바이스 환경에 최적화된 사용자 경험을 제공하기 위해 데스크탑, 모바일 각 기준에 맞춰 와이어프레임을 설계했습니다.



06. 피그마 프로토타이핑 (1)

다양한 디바이스 환경에 최적화된 사용자 경험을 제공하기 위해 데스크탑, 모바일 각 기준에 맞춰 와이어프레임을 설계했습니다.



1 인터랙션 설계

- 버튼 클릭 시
- :버튼 색이 바뀐다, 다른 페이지로 이동한다
- 호버효과
- :메뉴 박스 호버시 이미지 확대되는 애니
- 스크롤
- :배너영역을 내려가면 스크롤할 때 메뉴가 보임

2. 컴포넌트

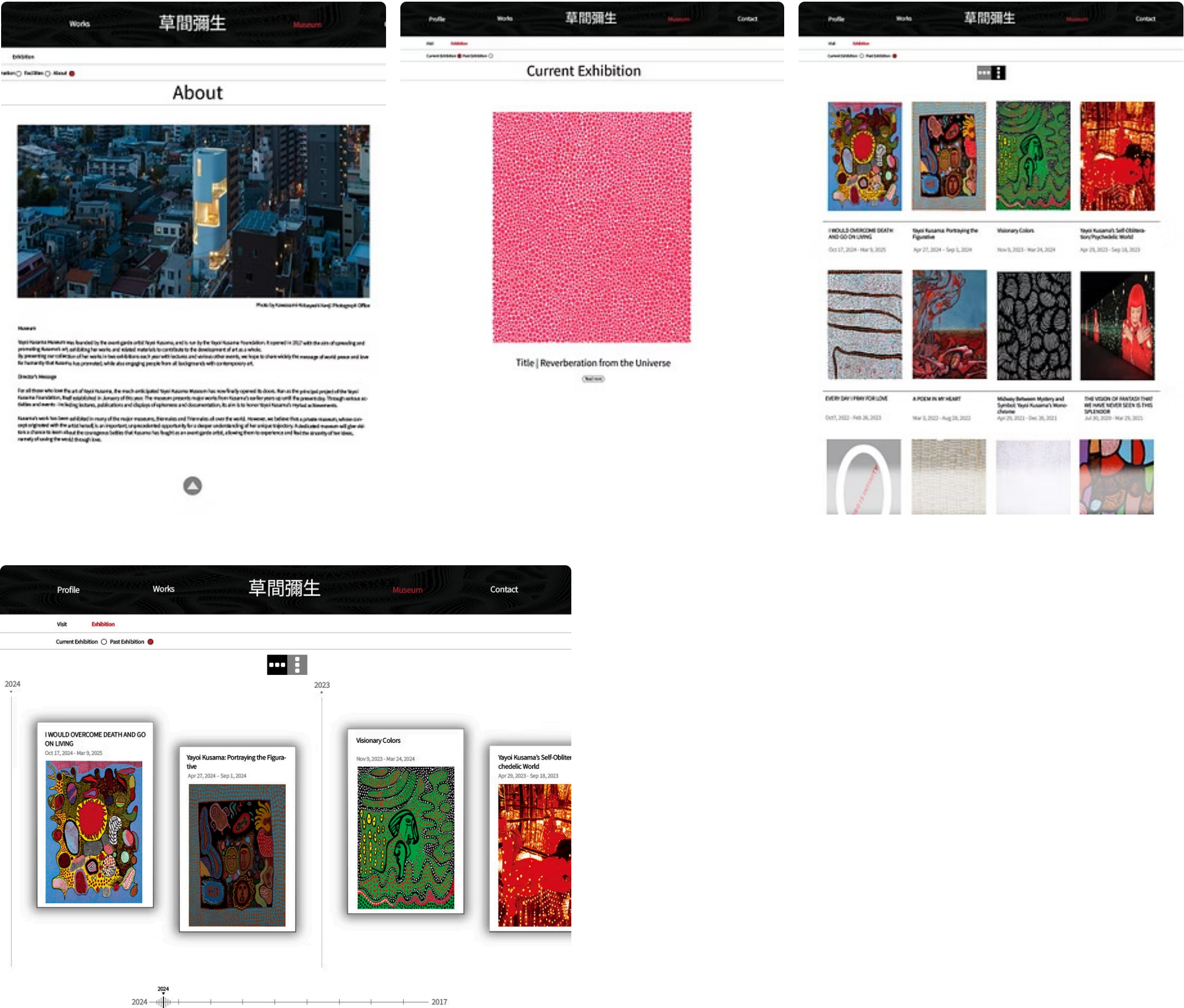
- 버튼 스타일
- 카드 UI
- 폼 요소
- 모달/팝업

3. 프로토타임

- 클릭플로우
- 인터랙션(Interaction) 설정
- 전환 효과(Transition & Animation)

06. 피그마 프로토타이핑 (2)

다양한 디바이스 환경에 최적화된 사용자 경험을 제공하기 위해 데스크탑, 모바일 각 기준에 맞춰 와이어프레임을 설계했습니다.



HTML 마크업

웹 표준과 접근성을 고려한 시맨틱 마크업으로 구조화된 콘텐츠를 제공합니다.

시맨틱 태그

<header>, <nav>, <main>, <section>, <article>, <footer> 태그를 활용하여 의미있는 문서 구조를 구성했습니다.

웹 접근성

<h1>~<h2> 계층 구조, alt 속성, video 접근성 속성을 적용하여 모든 사용자가 콘텐츠에 접근할 수 있도록 했습니다.

SEO 최적화

페이지 구조를 제목 태그로 의미있게 구분하고, 링크와 이미지에 적절한 anchor text와 alt 속성을 사용해 검색 엔진 최적화를 구현했습니다.

```
dy>
header class="top">
  <div id="top-area">...
</div>
<div id="banner-area">  <!-- 3. 작가배너영역 -->
  <section class="banner-area inbox"></section>
  <section class="scroll-inbox"></section>
</div>
/header>
div id="main-area">  <!-- 4. 메인영역 -->
  <div id="spart-menu">  <!-- 스티커서브메뉴 -->
    <nav class="spart-menu inbox">
      <div id="spart-menu2" class="center-box"> </div>
    </nav>
  </div> <!-- 상단동그라미 -->
  <div class="main-area circle"></div>
  <main id="maincontent" class="main-area inbox">...
</main>
<div id="contact-area">  <!-- 5. 컨택영역 -->
  <div class="contact-area inbox">
    <div class="paddingbox">
      <h2 class="temp-tit blind">컨택영역</h2>
      <div class="main-area circle2"></div>
      <h2>Contact</h2>
      <p>For requests and inquiries, please contact us below</p>
      <div class="contact-box">
        <article class="name-box desc-box">...
        </article>
        <article class="email-box desc-box">...
        </article>
        <article class="details-box desc-box">...
        </article>
        <article class="transmit-box desc-box">...
        </article>
      </div>
    </div>
  </div>
</div>
```



styles.css

```
1  /* 1. 상단 영역 레이아웃 */
2  .top {
3      position: sticky;
4      top: 0;
5      left: 0;
6      height: 100vh;
7      height: 100dvh;
8      z-index: -1;
9  }
10
11 .top-area {
12     position: sticky;
13     background: url(../images/background.gif) no-repeat center top / cover;
14     padding: 0 0 2.7vw;
15     width: 100%;
16 }
17
18 .top-area .inbox {
19     position: fixed;
20     height: 125px;
21     margin-bottom: 40px;
22     padding: 24px 0 24px 0;
23 }
24
25 /* 2. 메인 메뉴 호버 애니메이션 */
26 .contact:hover img {
27     animation: menu-ani 2s linear forwards;
28 }
29
30 @keyframes menu-ani {
31     to {
32         padding: 0;
33         width: 100%;
34         height: 100%;
35     }
36 }
```

CSS 스타일링

주요 기술

- **레이아웃:** Flexbox와 Grid 12 System으로 유연한 구조 설계
- **포지셔닝:** sticky, fixed를 활용한 동적 UI 배치
- **애니메이션:** keyframes와 hover 효과로 생동감 있는 인터랙션 구현

방법론

- **모듈화:** 역할별로 파일을 분리하여 체계적 관리
- **컴포넌트 기반:** 독립적이고 재사용 가능한 UI 블록 설계
- **시맨틱 네이밍:** 의미있는 클래스명으로 코드 가독성 향상

반응형 웹 구현

다양한 디바이스 환경에서 최적의 사용자 경험을 제공하기 위해 4개의 주요 브레이크포인트 (1360px, 1200px, 1024px, 800px)를 설정하여 화면 크기별로 레이아웃을 세밀하게 조정했습니다.

01

미디어쿼리 적용

@media screen을 활용해 각 브레이크포인트별 스타일을 분리하고 구간별 최적화를 진행했습니다.

02

유연한 단위 시스템

vw, vh, %, rem 등 상대 단위를 활용하여 화면 크기에 따라 자연스럽게 조정되는 레이아웃을 구현했습니다.

03

플렉스박스 재정렬

모바일 메뉴에서 display: flex와 flex-direction을 활용해 구조를 동적으로 재배치했습니다.

04

반응형 이미지

aspect-ratio, object-fit: cover, width: 100%로 모든 해상도에서 최적의 이미지 표현을 구현했습니다.

테스트 전략: 뷰포트 기반 단위와 구간별 미디어쿼리 분리를 통해 각 환경에서의 정확한 테스트가 가능하도록 구조화했습니다.

```
/* 1360px 이하 미디어쿼리 시작 */
> @media screen and (max-width: 1360px) { ...
}
/* 1360px 이하 미디어쿼리 종료 */

/* 1200px 이하 미디어쿼리 시작 */
> @media screen and (max-width: 1200px) { ...
}
/* 1200px 이하 미디어쿼리 종료 */

/* 1024px 이하 미디어쿼리 시작 */
> @media screen and (max-width: 1024px) { ...
}
/* 1024px 이하 미디어쿼리 종료 */

/* 800px 이하 미디어쿼리 시작 */
@media screen and (max-width: 800px) {
  /* //////////화면전체*/
  /* 모바일용 메뉴 */

  /* 메뉴그룹 변경 */
  .menu-group {
    height: 100%;
    display: flex;
    flex-direction: column;
    /* padding: 5vw; */
    background-color: white;
    justify-content: flex-start;
    align-items: flex-start;
    box-sizing: border-box;
    /* padding: 10vh 5vw; */
  }
}
```

JavaScript 구현 - 공통 부분 (1)

현재 페이지 상태에 따라 내비게이션 메뉴를 자동으로 활성화하는 인터랙티브 시스템의 주요 구현 기능을 요약합니다.

```
44 const EXCLUDE_TEXT = "草間彌生";
45 // 1. 현재 페이지의 URL 정보 가져오기
46 let currentPath = window.location.pathname.split('/').pop();
47 const currentHash = window.location.hash;
48 // 2. 모든 활성화 대상 요소를 찾습니다.
49 const navlinks = document.querySelectorAll('.line1 a, .spart-menu a, .line2 label, .change a');
50 // *****
51 // ★ 메뉴 그룹 경로 정의 ★
52 const WORKS_PAGES = ['sub2-1.html', 'sub2-1.html'];
53 const MUSEUM_PAGES = [
54   'sub3-1-1visitinfo.html', 'sub3-1-2Facilities.html', 'sub3-1-3about.html',
55   'sub3-2-1exh.html', 'sub3-2-2-1exh.html', 'sub3-2-2-2exh.html',
56 ];
57 // Exhibition (line1) 활성화 페이지
58 const EXHIBITION_PAGES = [
59   'sub3-2-1exh.html', 'sub3-2-2-1exh.html', 'sub3-2-2-2exh.html'
60 ];
61 // Past Exhibition (line2) 라벨 활성화 페이지
62 const PAST_EXHIBITION_PAGES = [
63   'sub3-2-2-1exh.html',
64   'sub3-2-2-2exh.html'
65 ];
66 // *****
```

```
67 // *****
68 navlinks.forEach(item => {
69   let linkhref = null;
70   let linkpath = null;
71   let linktext = item.textContent.trim();
72   let targetElement = item; // active 클래스를 추가할 최종 요소
73
74   // 1. 요소 타입에 따른 href/data-url 및 대상 설정
75   if (item.tagName === 'A') {
76     linkhref = item.getAttribute('href');
77     linkpath = linkhref ? linkhref.split('/').pop().split('*')[0] : null;
78
79     // ▲ 제외 대상 링크 처리
80     if (linktext === EXCLUDE_TEXT) {
81       item.closest('li').classList.remove('on');
82       item.classList.remove('active');
83       return;
84     }
85   } else if (item.tagName === 'LABEL') {
86     // LABEL 태그는 data-url 속성을 사용
87     linkhref = item.getAttribute('data-url');
88     linkpath = linkhref ? linkhref.split('/').pop().split('*')[0] : null;
89     // LABEL의 경우, targetElement에 active 클래스를 부여합니다.
90   } else {
91     return; // 그 외의 태그는 무시
92   }
93
94   // 2. 초기화 (혹시 모든 경우를 대비)
95   targetElement.classList.remove('active');
96   if (item.closest('li')) {
97     item.closest('li').classList.remove('on');
98   }
99   if (linkhref) return;
100 }
```

1

기능 설명: 현재 페이지 파일명(currentPath)과 해시(currentHash)를 가져옴

관련 코드: window.location.pathname,
window.location.hash

2

기능 설명: 메뉴 요소 전체를 탐색

관련 코드:
document.querySelectorAll('.line1 a, .spart-menu a, .line2 label, .change a')

3

기능 설명: 특정 텍스트("草間彌生")를 가진 메뉴는 활성화 제외

관련 코드: EXCLUDE_TEXT 처리

❏ **핵심 기능:** 페이지 그룹별 활성화 로직, 예외 처리(특정 작가명 배제), 해시 링크 처리, 실시간 섹션 감지로 사용자의 현재 위치를 명확하게 표시합니다.

JavaScript 구현 - 공통 부분 (2)

현재 페이지 상태에 따라 내비게이션 메뉴를 자동으로 활성화하는 인터랙티브 시스템의 주요 구현 기능을 요약합니다

```
-----
★ 개별 메뉴 및 뷰 모드 활성화 로직 (파일명 일치) ★
-----

F. 현재 페이지 파일명과 링크/데이터 url 파일명이 정확히 일치하는 경우 활성화
(Current Exhibition 라벨이나 일반 Works 메뉴 등이 여기서 처리됩니다)
(linkPath === currentPath) {
  targetElement.classList.add('active');

  // ★ .change 뷰 모드 버튼 처리 ★
  if (item.closest('.change')) {
    item.closest('li')?.classList.add('active');
  }

  // 상위 메뉴라면 <li>에도 'on' 클래스스 추가
  else if (item.closest('.spart-menu') && item.tagName === 'A') {
    item.closest('li')?.classList.add('on');
  }
}

G. 해시(#) 링크 처리 (Contact)
se if (linkPath === currentPath && linkHref.endsWith(currentHash) && currentHash !== '') {
  targetElement.classList.add('active');
  item.closest('li')?.classList.add('on');
}

H. 해시만 있는 링크 처리
se if (linkHref.startsWith('#') && linkHref === currentHash) {
  targetElement.classList.add('active');
  item.closest('li')?.classList.add('on');
}

I. Contact 등 해시(#) 링크 클릭 시 클래스 변경되도록 처리
ent.querySelectorAll("a[href~='#']").forEach(anchor => {
  anchor.addEventListener('click', function() {
    // 모든 a에서 active 제거
    document.querySelectorAll('.line1 a, .spart-menu a').forEach(a => a.classList.remove('active'));

    // 클릭한 a에 active 부여
    this.classList.add('active');
  });
});
```

```
-----
// ★ 그룹 메뉴 활성화 로직 ★
// -----

// A. Works 그룹 처리 (변경 없음)
const isWorksMenuLink = (linkPath === 'sub2.html' && linkText.toLowerCase() === 'works');
const isCurrentPageInWorksGroup = WORKS_PAGES.includes(currentPath);

if (isWorksMenuLink && isCurrentPageInWorksGroup) {
  if (item.tagName === 'A' && item.closest('.spart-menu')) {
    item.classList.add('active');
    item.closest('li')?.classList.add('on');
  }
}

// B. Museum 그룹 처리 (변경 없음)
const isMuseumMenuLink = (linkPath === 'sub3-1-1visitinfo.html' && linkText.toLowerCase() === 'museum');
const isCurrentPageInMuseumGroup = MUSEUM_PAGES.includes(currentPath);

if (isMuseumMenuLink && isCurrentPageInMuseumGroup) {
  item.classList.add('active');
  item.closest('li')?.classList.add('on');
}

// C. Visit 그룹 처리 (동위 line1) (변경 없음)
const isVisitMenuLink = (linkPath === 'sub3-1-1visitinfo.html' && linkText.toLowerCase() === 'visit');

if (isVisitMenuLink && (currentPath.startsWith('sub3-1-') && !currentPath.includes('exh'))) {
  item.classList.add('active');
}

// D. Exhibition 그룹 처리 (line1 - Exhibition)
// sub3-2-x 페이지 전체에서 line1 Exhibition 활성화
const isExhibitionMenuLink = (linkPath === 'sub3-2-1exh.html' && linkText.toLowerCase() === 'exhibition');
const isCurrentPageInExhibitionGroup = EXHIBITION_PAGES.includes(currentPath);

if (isExhibitionMenuLink && isCurrentPageInExhibitionGroup) {
  item.classList.add('active');
}

// -----
// ★ E. Past Exhibition 그룹 처리 (line2 - Past Exhibition 라벨) ★
// sub3-2-2-1exh.html, sub3-2-2-2exh.html 페이지에서 라벨 활성화
// -----
const isPastExhLabel = (item.tagName === 'LABEL' );
const isCurrentPageInPastExhGroup = PAST_EXHIBITION_PAGES.includes(currentPath);

if (isPastExhLabel && isCurrentPageInPastExhGroup) {
  item.classList.add('active');
}
```

4

기능 설명: 페이지 그룹(Works, Museum, Exhibition, Past Exhibition)에 따라 활성화 규칙 적용

관련 코드: WORKS_PAGES, MUSEUM_PAGES, EXHIBITION_PAGES, PAST_EXHIBITION_PAGES

5

기능 설명: 현재 페이지 파일명과 일치하는 링크 자동 활성화

관련 코드: if (linkPath === currentPath)

6

기능 설명: 해시(#) 기반 페이지 내 링크 (Contact 등) 클릭 시 실시간 활성화 반응

관련 코드: 마지막 부분의 addEventListener('click', ...)

7

기능 설명: 각 메뉴의 계층 구조(li)까지 활성화(on) 처리

관련 코드: item.closest('li')?.classList.add('on')

☒ **핵심 기능:** 페이지 그룹별 활성화 로직, 예외 처리(특정 작가명 배제), 해시 링크 처리, 실시간 섹션 감지로 사용자의 현재 위치를 명확하게 표시합니다.

JavaScript 구현 - 작품 페이지 (1)

페이지네이션, 작품 리스트, 모달 상세보기가 결합된 작품 아카이브 시스템입니다.

```
1 window.addEventListener("DOMContentLoaded", () => {
2   const wrap = document.querySelector(".wrap");
3   const pagingList = document.querySelectorAll(".dot");
4
5   const UNIT_NUM = 6;
6   const workTitle = [ ...
11  ];
12
13   // 페이지 초기값
14   let currentPage = 1;
15
16   // DOM 요소 캐시
17   const leftBtn = document.querySelector(".abtn.ab1");
18   const rightBtn = document.querySelector(".abtn.ab2");
19   const dotsContainer = document.querySelector(".paging-list .dots");
20
21   // totalPages 계산
22   const totalPages = Math.max(1, Math.ceil(workTitle.length / UNIT_NUM));
23
24   // 동적으로 dot이 필요하면 렌더링 (선택; 이미 HTML에 dot이 있으면 이 블록은 건너뛰어도 됩니다)
25   function renderDotsIfNeeded() {
26     const existingDots = dotsContainer.querySelectorAll('.dot');
27     if (existingDots.length === totalPages) return; // 이미 있음
28
29     dotsContainer.innerHTML = '';
30     for (let i = 1; i <= totalPages; i++) {
31       const span = document.createElement('span');
32       span.className = 'dot';
33       span.dataset.val = String(i);
34       dotsContainer.appendChild(span);
35     }
36   }
```

```
37 // dot 스타일 업데이트 (활성화 표시)
38 Windsurf: Refactor | Explain | X
39 function updateDots(page) {
40   const dots = dotsContainer.querySelectorAll('.dot');
41   dots.forEach(dot => {
42     const val = Number(dot.dataset.val);
43     if (val === page) {
44       dot.style.backgroundColor = '#666666';
45       dot.classList.add('active');
46     } else {
47       dot.style.backgroundColor = '';
48       dot.classList.remove('active');
49     }
50   });
51
52   // 좌/우 버튼 활성화/비활성화(선택) 처리: 끝이면 비활성화 시키기
53   if (leftBtn) leftBtn.classList.toggle('disabled', page <= 1);
54   if (rightBtn) rightBtn.classList.toggle('disabled', page >= totalPages);
55 }
56
57 // 페이지 이동 함수 (MakeWorkList 호출 + 상태 업데이트)
58 Windsurf: Refactor | Explain | X
59 function goToPage(page) {
60   // 범위 검사
61   if (page < 1) page = 1;
62   if (page > totalPages) page = totalPages;
63
64   currentPage = page;
65   // 실제 리스트 렌더링
66   MakeWorkList(currentPage);
67
68   // dot 상태 업데이트
69   updateDots(currentPage);
70 }
```

```
71 // 왼쪽/오른쪽 버튼 이벤트 바인딩
72 if (leftBtn) {
73   leftBtn.addEventListener('click', () => {
74     if (currentPage > 1) goToPage(currentPage - 1);
75   });
76 }
77 if (rightBtn) {
78   rightBtn.addEventListener('click', () => {
79     if (currentPage < totalPages) goToPage(currentPage + 1);
80   });
81 }
82
83 // dots 클릭 바인딩 (delegation 또는 개별 바인딩)
84 Windsurf: Refactor | Explain | X
85 function bindDotClicks() {
86   // 이벤트 위임: dotsContainer에 클릭될지
87   dotsContainer.addEventListener('click', (e) => {
88     const dot = e.target.closest('.dot');
89     if (!dot) return;
90     const page = parseInt(dot.dataset.val, 10);
91     if (Number.isNaN(page)) return;
92     goToPage(page);
93   });
94 }
95
96 // 초기화 (페이지 수에 따라 dot 렌더링/바인딩 후 첫 렌더 호출)
97 renderDotsIfNeeded();
98 bindDotClicks();
99 goToPage(1); // 초기 페이지 렌더링
```

주요 구현 기능

1. **작품 리스트 출력:** 페이지당 6개 작품을 동적으로 표시
2. **Dot 페이지네이션:** 작품 개수에 따라 자동 생성되는 페이지 점
3. **페이지 이동:** 좌우 버튼 또는 dot 클릭으로 전환


```

269
270 // 4. 모달 기능
271 const mWindow = document.getElementById("modal");
    Windsurf: Refactor | Explain | Generate JSDoc | X
272 function openModal(e) {
273     let idx = e.currentTarget.parentElement.getAttribute('data-num');
274     console.log(idx);
275     // 배열.some((v)=>{if(조건){실행}})
276     // some은 배열을 순회하면서 조건에 맞으면
277     // 실행문을 실행하고 return true를 쓰면 끝마친다!
278     workInfo.some(v=>{
279         if(v.idx == idx){
280             console.log(v.title);
281             mWindow.querySelector('h3').textContent = v.title;
282             mWindow.querySelector('.date').textContent = v.date;
283             mWindow.querySelector('.technique').textContent = v.technique;
284             mWindow.querySelector('.dimensions').textContent = v.dimensions;
285             return true;
286         }
287
288         console.log(v.title);
289
290     })
291     mWindow.style.display = "block";
292     mWindow.querySelector('.modal-img img').src = `./images/paint${idx}.jpg`;
293     document.body.style.overflow = "hidden"; // 바디스크롤막기효과
294 }
295
296

```

JavaScript 구현 - 작품 페이지 (2)

주요 구현 기능

4. 모달 상세보기: 작품 클릭 시 이미지, 제목, 제작연도, 기법 정보 표시

5. 스크롤 제어: 모달 오픈 시 body 스크롤 방지

핵심 기술

데이터 관리: workTitle과 workInfo 배열로 구조적 데이터 관리

최적화: 조건부 렌더링과 배열 탐색 최적화로 효율적 검색 구현

반응형 UX: display와 overflow 제어로 상태별 시각적 전환

JavaScript 구현 - 층 페이지

층별 버튼 클릭 시 해당 층의 지도 이미지와 설명을 전환하는 인터랙티브 층 안내 시스템입니다.

- 라벨 기반 페이지 이동

data-url 속성을 읽어 해당 페이지로 이동하고 라디오 버튼 체크 상태를 동기화합니다.

- 자동 페이지 인식

현재 URL과 label 속성을 비교하여 페이지 로드 시 자동으로 해당 층을 선택합니다.

- 층별 상태 관리

선택층은 활성 이미지로, 위층은 숨김으로, 아래층은 흐림 처리하여 시각적 계층을 구현했습니다.

데이터 속성 활용

data-url, data-floor 속성으로 층 정보와 페이지 정보를 HTML에 저장하여 JS에서 효율적으로 조회합니다.

배열 기반 관리

floorsOrder와 mapPos 배열로 층 순서와 지도 이동 위치를 체계적으로 관리하여 유지보수성을 향상시켰습니다.

```
floor_map_compact.js

1 "keyword">const DEFAULT_FLOOR = "5F";
2 "keyword">const floorsOrder = ["5F", "4F", "3F", "2F", "1F", "B1F"];
3 "keyword">const mapPos = [18, 8, 0, -8, -16, -9];
4
5 "keyword">const floorItems = document.querySelectorAll(".floortxt");
6 "keyword">const mapWrap = document.querySelector(".mapwrap");
7 "keyword">const maps = document.querySelectorAll(".mapwrap > div");
8
9 "keyword">let currentOpenFloor = null;
10
11 "keyword">const floorImages = {
12   "5F": { default: "./images/gallery1.png", active: "./images/gallery1.png" },
13   "4F": { default: "./images/4f_down.png", active: "./images/gallery2.png" },
14   "3F": { default: "./images/3f_down.png", active: "./images/gallery3.png" },
15   "2F": { default: "./images/2f_down.png", active: "./images/gallery4.png" },
16   "1F": { default: "./images/1f_down.png", active: "./images/gallery5.png" },
17   "B1F": { default: "./images/gallery6.png", active: "./images/gallery6.png" }
18 };
19
20 "keyword">function updateMaps(activeFloor) {
21   "keyword">const index = floorsOrder.indexOf(activeFloor);
22   mapWrap.style.translate = `0 ${mapPos[index]}%`;
23
24   maps.forEach((m) => {
25     "keyword">const i = floorsOrder.indexOf(m.dataset.floor);
26     "keyword">const floor = m.dataset.floor;
27     "keyword">const imgEl = m.querySelector("img");
28
29     "keyword">if (i < index) {
30       imgEl.src = floorImages[floor].default;
31       Object.assign(m.style, { opacity: 0, visibility: "hidden", zIndex: 0, pointerEvents: "none" });
32     } "keyword">else "keyword">if (i === index) {
33       imgEl.src = floorImages[floor].active;
34       Object.assign(m.style, { opacity: 1, filter: "none", zIndex: 99, visibility: "visible", pointerEvents: "auto" });
35     } "keyword">else {
36       imgEl.src = floorImages[floor].default;
37       Object.assign(m.style, { opacity: 0.3, filter: "blur(2px)", visibility: "visible", zIndex: 0, pointerEvents: "none" });
38     }
39   });
40 }
41
42 "keyword">function setActive(floor) {
43   floorItems.forEach((li) => {
44     li.classList.toggle("active", li.dataset.floor === floor);
45   });
46
47   "keyword">if (floor === currentOpenFloor) "keyword">return;
48   updateMaps(floor);
49   currentOpenFloor = floor;
50 }
51
52 floorItems.forEach((li) => {
53   li.addEventListener("click", () => setActive(li.dataset.floor));
54 });
55
56 setActive(DEFAULT_FLOOR);
```

JavaScript 구현 - 전시 페이지

화면 스크롤 중 특정 요소(.bottom-area)가 뷰포트에 진입하는 순간을 감지하여 <main> 요소에 .end 클래스를 추가하거나 제거하는 시각적 상태 전환 기능입니다.



실시간 스크롤 감지

scroll 이벤트로 window.scrollY와 요소의 getBoundingClientRect().top을 지속적으로 비교합니다.



뷰포트 진입 판단

요소의 top 좌표가 window.innerHeight보다 작아지면 화면에 들어온 것으로 판단하여 .end 클래스를 추가합니다.



CSS 트리거 역할

.end 클래스가 배경색 전환, 페이드인 효과, 고정 헤더 해제 등의 스타일 변화를 촉발합니다.

활용 기술 스택

- 순수 JavaScript (ES6)
- document.querySelector() DOM 탐색
- getBoundingClientRect() 위치 계산
- addEventListener('scroll') 이벤트 제어
- classList API 상태 전환

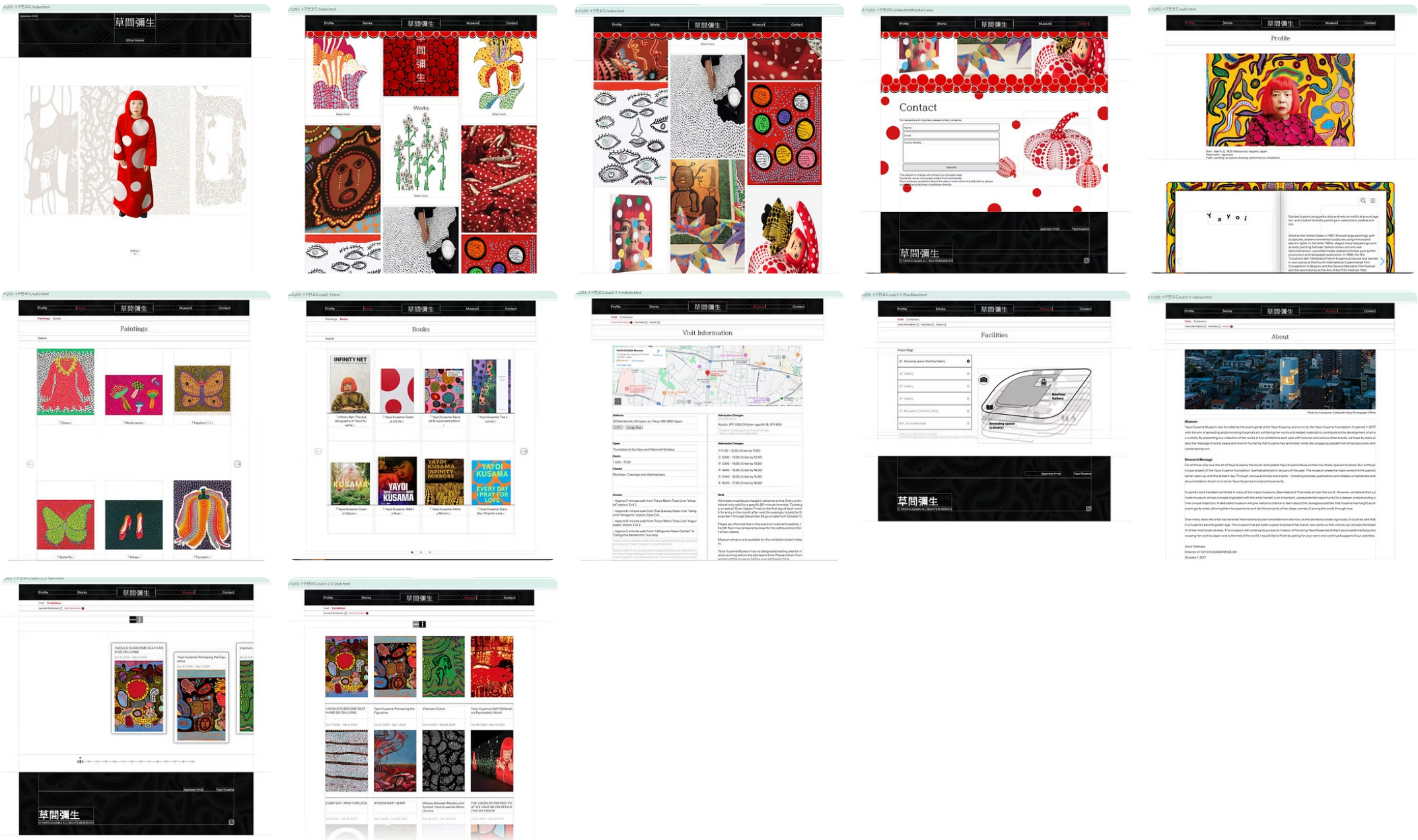
```
Windsurf: Refactor | Explain | ✕
const getBCR = (el) => el.getBoundingClientRect().top;

// 타겟박스
const target = document.querySelector('.bottom-area');
// 변경박스
const chgBox = document.querySelector('main');
// 윈도우 높이값 (기준값)
let winH = window.innerHeight;

window.addEventListener('scroll', () => {
  let scTop = window.scrollY;
  let tgPos = getBCR(target);
  console.log('스크롤이동중', scTop, tgPos);
  if(tgPos < winH) {
    chgBox.classList.add('end');
  }
  else {
    chgBox.classList.remove('end');
  }
});
```

프로젝트 결과물

최종적으로 10개 페이지의 완전한 반응형 웹사이트를 구현했습니다. 각 페이지는 웹 표준과 접근성을 준수하며, 다양한 JavaScript 인터랙션으로 향상된 사용자 경험을 제공합니다.



10

페이지 퍼블리싱

완전한 반응형 웹 디자인 구현

4

브레이크포인트

다양한 디바이스 최적화

개선 방향

미디어쿼리 보완: 더 세밀한 브레이크포인트 추가로 다양한 디바이스 대응

새로운 페이지 생성: 추가 콘텐츠 페이지 개발 및 기능 확장